Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA

(Deemed To Be University)

-

Project – Low Level Design
on

Cloud Cost Intelligence Platform for Smart City

Student Name:

Malavika Rajesh

23BCAICD050

Industry Mentor:
Mr.Shashank

Guided by:
Mrs. Niharika

Table of Contents

1. Introduction

1.1 Purpose
1.2 Scope
1.3 Definitions and Abbreviations

2. Module Design

2.1 Application Structure
2.2 Component Overview
2.3 Function Responsibilities
2.4 System Architecture

3. Detailed Function Specifications

3.1 index()
3.2 analyze_costs()
3.3 create_chart_image()
3.4 _fig()
3.5 Chart Generation Functions

4. Data Flow

4.1 Request Lifecycle
4.2 Data Processing Flow

5. Template Design

5.1 Page Structure
5.2 UI Components
5.3 CSS & Styling

6. Error Handling

6.1 Validation Rules
6.2 Exception Handling

7. Module-Level Constants

8. Dependency Catalogue

9. Testing Considerations

9.1 Unit Test Cases
9.2 Sample Input Data

10. References

1. Introduction

1.1 Purpose

This Low Level Design (LLD) document provides a detailed, implementation-level description of the CloudLens application. It expands upon the High Level Design (HLD) by specifying exact function signatures, class structures, variable names, algorithm logic, data validation rules, and inter-module interactions as reflected directly in the source code (app.py).

1.2 Scope

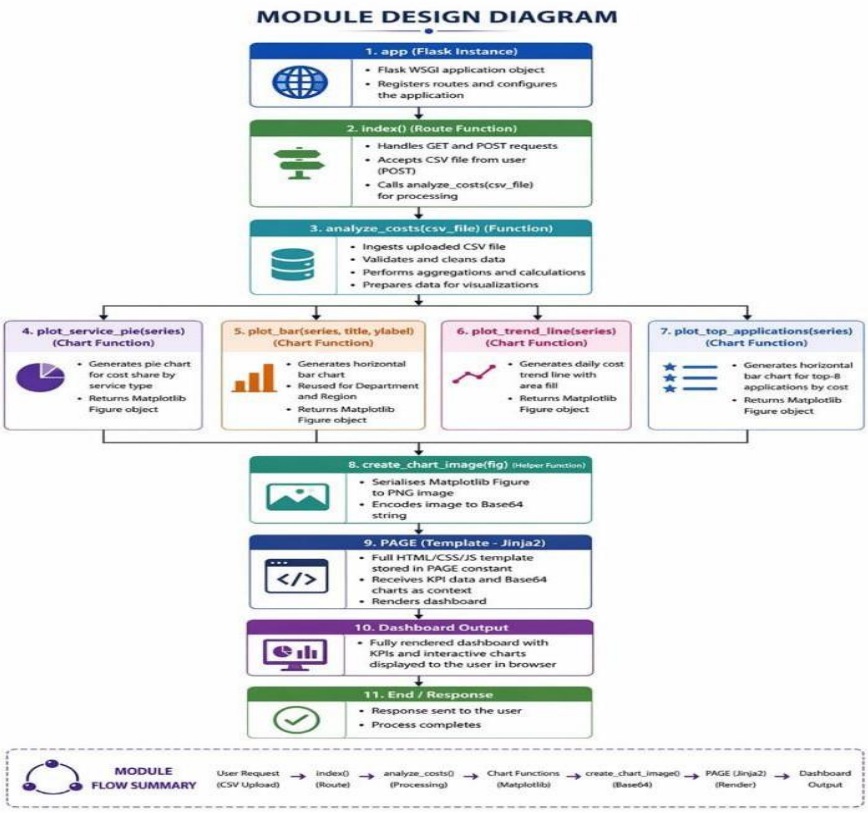
The LLD covers the following modules present in the single-file Flask application:

- Flask route handler (index)
- Data ingestion and analysis module (analyze_costs)
- Chart generation module — five dedicated plot functions
- Chart serialisation helper (create_chart_image)
- Jinja2 HTML/CSS template (PAGE constant)

1.3 Definitions and Abbreviations

Term	Definition
CSV	Comma-Separated Values — the input file format accepted by the application
DataFrame (df)	In-memory tabular data structure provided by the pandas library
Base64	Binary-to-text encoding used to embed PNG chart images inline in HTML
BytesIO	Python in-memory byte stream; used to avoid writing chart files to disk
DXA	Unit used in DOCX formatting (1 inch = 1440 DXA)
KPI	Key Performance Indicator — summary metric shown in the dashboard cards
INR	Indian Rupee — the cost unit used throughout the data model
MVC	Model-View-Controller — the architectural pattern used by Flask
Jinja2	Python templating engine integrated with Flask

2. Module Design



Module / Symbol	Type	Responsibility
app	Flask instance	WSGI application object; registers routes
PAGE	str constant	Full HTML/CSS/JS template rendered by Jinja2
index()	Route function	Handles GET and POST requests; orchestrates analysis and rendering
analyze_costs(csv_file)	Function	Ingests, validates, and aggregates the uploaded CSV
create_chart_image(fig)	Helper function	Serialises a Matplotlib Figure to a base64 PNG string
plot_service_pie(series)	Chart function	Generates a pie chart of cost share by service type
plot_bar(series, title, ylabel)	Chart function	Generic horizontal bar chart (reused for Department and Region)
plot_trend_line(series)	Chart function	Generates the daily cost trend line with area fill
plot_top_applications(se ries)	Chart function	Horizontal bar chart for top-8 applications by cost
_fig(w, h)	Helper function	Factory for styled Matplotlib (fig, ax) tuples with dark theme

3. Detailed Function Specifications

3.1 index()

3.1.1 Signature

```
def index() -> flask.Response
```

3.1.2 Route Binding

```
@app.route("/", methods=["GET", "POST"])
```

3.1.3 Local Variables

Variable	Type	Initial Value	Purpose
filename	str | None	None	Stores the original filename of the uploaded CSV for display
error	str | None	None	Holds a user-facing error message if processing fails
kpis	dict | None	None	Formatted KPI strings passed to the Jinja2 template
charts	dict	{}	Maps chart names to base64 PNG strings for template embedding

3.1.4 Logic Flow

- GET request: returns render_template_string(PAGE) with all optional variables set to None / empty dict — renders blank dashboard.
- POST request:
  - Retrieves file from request.files.get('myfile').
  - If no file or filename is empty: sets error = 'No file chosen.' and falls through to render.
  - Otherwise: calls analyze_costs(file), populates kpis dict with formatted strings (f'{value:,}'), generates five charts by calling each plot function and passing the result to create_chart_image().
  - Any Exception raised inside the try block is caught; error is set to f'Could not analyse file — {exc}'.
- Returns render_template_string(PAGE, filename=filename, error=error, kpis=kpis, charts=charts).

3.1.5 KPI Dictionary Schema

Key	Source Expression	Format
total_cost	analysis['total_cost']	f'{value:,}' — comma-separated integer string
avg_daily_cost	analysis['avg_daily_cost']	f'{value:,}'
top_department_name	analysis['top_department_name']	Plain string
top_department_cost	analysis['top_department_cost']	f'{value:,}'
top_application_name	analysis['top_application_name']	Plain string
top_application_cost	analysis['top_application_cost']	f'{value:,}'

3.2 analyze_costs(csv_file)

3.2.1 Signature

```
def analyze_costs(csv_file: FileStorage) -> dict
```

3.2.2 Input

csv_file — a Werkzeug FileStorage object (file-like stream) received from request.files.

3.2.3 Validation Rules

Check	Method	Exception Raised
'Date' column present	Column membership check	ValueError: 'CSV must include a Date column.'
Date values parseable	pd.to_datetime(errors='coerce') + isna().all()	ValueError: 'Could not parse any Date values.'
'Cost_INR' column present	Column membership check	ValueError: 'CSV must include a Cost_INR column.'
Required categorical columns present	df.dropna(subset=[...])	Silently drops incomplete rows; no exception

3.2.4 Processing Steps (in order)

- Read CSV: df = pd.read_csv(csv_file)
- Parse dates: df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
- Coerce costs: df['Cost_INR'] = pd.to_numeric(df['Cost_INR'], errors='coerce').fillna(0)

- Drop nulls: df = df.dropna(subset=['Department', 'Service_Type', 'Application', 'Region', 'Date'])
- Compute aggregations — see table below.

3.2.5 Aggregations

Output Key	Pandas Expression	Type
total_cost	int(df['Cost_INR'].sum())	int
avg_daily_cost	int(df.groupby('Date')['Cost_INR'].sum().mean())	int
top_department_name	department_cost.index[0]	str
top_department_cost	int(department_cost.iloc[0])	int
top_application_name	top_application_costs.index[0]	str
top_application_cost	int(top_application_costs.iloc[0])	int
department_cost	df.groupby('Department')['Cost_INR'].sum().sort_values(ascending=False)	pd.Series
region_cost	df.groupby('Region')['Cost_INR'].sum().sort_values(ascending=False)	pd.Series
service_costs	df.groupby('Service_Type')['Cost_INR'].sum().sort_values(ascending=False)	pd.Series
cost_trend	df.groupby('Date')['Cost_INR'].sum().sort_index()	pd.Series (DatetimeIndex)
top_application_costs	df.groupby('Application')['Cost_INR'].sum().sort_values(ascending=False).head(8)	pd.Series

3.3 create_chart_image(fig)

3.3.1 Signature

```
def create_chart_image(fig: matplotlib.figure.Figure) -> str
```

3.3.2 Algorithm

- Create an in-memory buffer: buf = BytesIO()
- Save the figure: fig.savefig(buf, format='png', bbox_inches='tight', facecolor=BG)
- Seek to start: buf.seek(0)
- Encode: data = base64.b64encode(buf.read()).decode('utf-8')
- Close figure to free memory: plt.close(fig)
- Return data — the caller embeds it as src="{data}" in the HTML <img> tag.

3.4 _fig(w=6, h=4.2)

3.4.1 Signature

```
def _fig(w: float = 6, h: float = 4.2) -> tuple[Figure, Axes]
```

3.4.2 Behaviour

Factory function that creates a consistently styled (fig, ax) pair. Applies the dark theme by setting figure and axes background colours, spine colours, tick label colours, and grid style using the module-level colour constants.

Constant	Hex Value	Applied To
BG	#08090b	Figure and axes facecolor; chart image background
SURFACE	#0e1117	Axes facecolor (plot area)
TEXT	#e8eaf0	Tick labels, axis titles, chart titles
MUTED	#6b7280	Axis label text, tick mark colours
GRID	#1a1e28	Grid lines and spine border colour

3.5 Chart Generation Functions

3.5.1 plot_service_pie(series)

Attribute	Value
Input	pd.Series — index: service type labels; values: cost totals
Figure size	5 × 4.2 inches, 110 DPI (created independently, not via _fig())
Plot type	Pie chart via ax.pie()
Palette	['#5b8af7','#34d399','#f59e0b','#a78bfa','#f87171','#22d3ee','#fb923c','#e879f9']
wedgeprops	linewidth=1.2, edgecolor=BG
pctdistance	0.78 (percentage labels placed at 78% of radius)
startangle	140 degrees
autopct	'%1.1f%%'
Title	'Service share'

3.5.2 plot_bar(series, title, ylabel)

Attribute	Value
Input	pd.Series (sorted descending); title: str; ylabel: str
Figure size	6 × 4.2 inches via _fig()
Plot type	Horizontal bar chart via ax.barh() — series reversed for top-to-bottom display
Colour scheme	Top bar: '#5b8af7' (accent blue); remaining bars: '#2d4a8a' (dark blue)
Bar height	0.62
Grid	x-axis only (color=GRID, linewidth=0.6); y-axis grid hidden
Used for	Department breakdown and Regional distribution (both reuse this function)

3.5.3 plot_trend_line(series)

Attribute	Value
Input	pd.Series with DatetimeIndex and daily cost values
Figure size	7 × 4 inches via _fig()
Plot type	Line plot + area fill + scatter markers
Line	color='#5b8af7', linewidth=1.8, solid_capstyle='round'
Fill	ax.fill_between() with alpha=0.12, color='#5b8af7'
Markers	ax.scatter() with s=28, edgecolors=BG, linewidths=1.2, zorder=5
Date formatting	fig.autofmt_xdate(rotation=30)

3.5.4 plot_top_applications(series)

Attribute	Value
Input	pd.Series — top 8 applications by cost (pre-sliced in analyze_costs)
Figure size	6 × 4.2 inches via _fig()
Plot type	Horizontal bar chart via ax.barh()
Colour scheme	Top bar: '#34d399' (green); remaining: '#1a6649' (dark green)
Bar height	0.62

4. Data Flow — Request Lifecycle

The following table traces every transformation a data point undergoes from the moment the CSV is submitted to the rendered HTML response.

Steps	Operation	Input Type	Output Type	Key Function / Library
1	Browser submits form	HTML multipart/form-data	HTTP POST body	Browser
2	Flask receives file stream	HTTP POST body	FileStorage object	flask.request.files
3	CSV parsed into DataFrame	FileStorage stream	pd.DataFrame	pd.read_csv()
4	Date column parsed	String column	datetime64 column	pd.to_datetime()
5	Cost column coerced	Mixed string/numeric	float64 column	pd.to_numeric()
6	Null rows dropped	DataFrame with NaNs	Clean DataFrame	df.dropna()
7	Aggregation computed	Clean DataFrame	pd.Series objects	df.groupby().sum()
8	KPI values extracted	pd.Series	int / str scalars	series.index[0], series.iloc[0]
9	Matplotlib figure created	pd.Series	Figure object	plot_* functions + _fig()
10	Figure saved to buffer	Figure object	PNG bytes (BytesIO)	fig.savefig()
11	Buffer encoded	PNG bytes	base64 str	base64.b64encode()
12	Template rendered	base64 str + KPI dict	HTML string	render_template_string()
13	HTML returned to browser	HTML string	HTTP 200 Response	Flask

5. Template Design (PAGE Constant)

5.1 Structure

The PAGE constant is a single multi-line Python string containing a complete HTML5 document. It is rendered by Jinja2 through render_template_string(). The template is divided into the following sections:

Template Section	Jinja2 Variables Used	Purpose
Top bar	None	Brand identity and application title
Page heading	None	Static title and description
Upload card	filename	File input form; displays last uploaded filename
Error block	error	Conditionally rendered; shown only when {% if error %}
KPI row (4 cards)	kpis.total_cost, kpis.avg_daily_cost, kpis.top_department_name, kpis.top_department_cost, kpis.top_application_name, kpis.top_application_cost	Displays key metric cards; shows '—' placeholder when kpis is None
Chart: Daily Trend	charts.trend_line	Wide (span-8) line chart; renders <img> with base64 src or empty-state div
Chart: Service Pie	charts.service_pie	Narrow (span-4) pie chart
Chart: Department Bar	charts.department_bar	Narrow (span-4) horizontal bar
Chart: Region Bar	charts.region_bar	Narrow (span-4) horizontal bar
Chart: Top Apps	charts.top_app_bar	Narrow (span-4) horizontal bar
Inline JavaScript	None	updateName() — updates filename label on file selection

5.2 CSS Variable Reference

CSS Variable	Value	Usage
--bg	#08090b	Page background; chart image background
--surface	#0e1117	Card and chart panel background
--surface2	#141821	Secondary surface (upload icon background)
--border	rgba(255,255,255,0.07)	Default card/panel border
--border-hi	rgba(255,255,255,0.13)	Upload zone and icon borders
--accent	#5b8af7	Primary interactive colour (button, file label, charts)
--accent-dim	rgba(91,138,247,0.12)	Upload zone hover background
--green	#34d399	KPI trend label colour; top-apps chart
--amber	#f59e0b	Top department card accent
--red	#f87171	Error message colour
--muted	#6b7280	Secondary text, axis labels
--mono	'DM Mono', monospace	Monospace font for KPI labels, file names, sub-labels
--sans	'Sora', sans-serif	Primary UI font
--radius	14px	Default card border radius
--radius-lg	22px	Large card border radius (upload card, KPI cards, chart cards)

5.3 Responsive Breakpoints

Breakpoint	CSS Change
max-width: 1024px	KPI row switches to 2-column grid; all chart cards expand to full-width (span 12)
max-width: 640px	Shell padding reduced; KPI row becomes 2×2 grid; upload card stacks vertically; top bar stacks vertically

6. Error Handling

Scenario	Trigger	Handling Mechanism	User-Facing Output
No file submitted	file is None or filename == ''	Explicit check in index()	error = 'No file chosen. Please pick a CSV.'
Missing Date column	'Date' not in df.columns	ValueError raised in analyze_costs()	error = 'Could not analyse file — CSV must include a Date column.'
Unparseable dates	df['Date'].isna().all()	ValueError raised in analyze_costs()	error = 'Could not analyse file — Could not parse any Date values.'
Missing Cost_INR column	'Cost_INR' not in df.columns	ValueError raised in analyze_costs()	error = 'Could not analyse file — CSV must include a Cost_INR column.'
Malformed cost values	Non-numeric Cost_INR entries	pd.to_numeric(errors='coerce').fillna(0)	Silently set to 0; no error shown
Missing categorical columns	NaN in Department, Service_Type, etc.	df.dropna() removes affected rows	Silently drops rows; analysis continues
Any other unexpected exception	Unhandled exception inside try block	except Exception as exc: error = f'Could not analyse file — {exc}'	Generic error message surfaced in UI

7. Module-Level Constants

Constant	Value	Type	Purpose
BG	'#08090b'	str	Darkest background colour for charts
SURFACE	'#0e1117'	str	Axes area colour for charts
TEXT	'#e8eaf0'	str	Light text colour for chart labels
MUTED	'#6b7280'	str	Muted grey for axis labels and secondary text
GRID	'#1a1e28'	str	Grid line and spine colour
PAGE	Multi-line HTML string (~450 lines)	str	Full Jinja2 template for the dashboard

8. Dependency Catalogue

Library	Import Alias	Version Constraint	Usage in Code
Flask	flask	>=2.0	Flask(), app.route(), request, render_template_string
pandas	pd	>=1.3	read_csv, to_datetime, to_numeric, groupby, dropna, Series
matplotlib	matplotlib / plt	>=3.4	Figure, Axes, pyplot; all chart generation and serialisation
numpy	np	>=1.21	Imported but not directly called in current code (transitive dependency of pandas/matplotlib)
base64	(stdlib)	N/A	b64encode() — encodes PNG bytes to ASCII string
io.BytesIO	(stdlib)	N/A	In-memory binary buffer for PNG serialisation

9. Testing Considerations

9.1 Unit Test Cases

Test ID	Function Under Test	Input	Expected Output
UT-01	analyze_costs()	Valid CSV with all required columns	Dict with correct aggregated values
UT-02	analyze_costs()	CSV missing 'Date' column	ValueError with message 'CSV must include a Date column.'
UT-03	analyze_costs()	CSV with all invalid dates	ValueError with message 'Could not parse any Date values.'
UT-04	analyze_costs()	CSV missing 'Cost_INR' column	ValueError with message 'CSV must include a Cost_INR column.'
UT-05	analyze_costs()	CSV with non-numeric Cost_INR values	Rows with bad costs set to 0; no exception
UT-06	create_chart_image()	Valid Matplotlib Figure	Non-empty base64-encoded string
UT-07	index() — GET	HTTP GET /	200 response; charts and kpis absent from context

Test ID	Function Under Test	Input	Expected Output
UT-08	index() — POST (valid)	HTTP POST with valid CSV	200 response; charts dict has 5 keys; kpis dict has 6 keys
UT-09	index() — POST (no file)	HTTP POST with no file	200 response; error message set

9.2 Sample CSV Structure

A minimal valid test CSV must include the following headers and representative data:

```
Date,Cost_INR,Department,Service_Type,Application,Region
2024-01-01,12500,Engineering,Compute,AuthService,ap-south-1
2024-01-01,8300,Marketing,Storage,CampaignApp,us-eas
2024-01-02,15700,Engineering,Network,APIGateway,ap-south-1
```

10. References

#	Reference	URL
1	Flask Documentation	https://flask.palletsprojects.com/
2	Pandas User Guide	https://pandas.pydata.org/docs/user_guide/
3	Matplotlib Documentation	https://matplotlib.org/stable/contents.html
4	Jinja2 Template Designer Docs	https://jinja.palletsprojects.com/en/3.x/templates/
5	Python base64 module	https://docs.python.org/3/library/base64.html
6	Python io.BytesIO	https://docs.python.org/3/library/io.html